

AI ASSISTENT CODING - 15.4

Baisa Vaishnavi

2303A52142

Batch - 41

Task 1: Initial API Setup for a Service Platform

Scenario

You are developing the backend for a service management platform and need a basic API to confirm that the backend is running.

Task Description

- Use AI assistance to generate a basic Flask application.
- Create a root endpoint /status that returns a JSON message indicating the API is live.
- Run the server locally and verify the response using a browser or API testing tool.

Prompt:

Generate a basic Flask API with a /status endpoint returning JSON response.

Code:

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/status')
def status():
    return jsonify({"status": "ok", "message": "API is running!"})

if __name__ == '__main__':
    # Run the app locally. In Colab, you might need to use ngrok for external access.
    # For local testing, you can run this cell and access http://127.0.0.1:5000/status
    app.run(debug=True, port=5000)
```

Output:

```
... * Serving Flask app '__main__'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Justification:

- Confirms backend availability
- Baseline endpoint for health check

Task 2: Create API to Add and View Resources

Scenario

Your backend needs to manage customer service requests stored temporarily in memory.

Task Description

- Use AI to help implement:
 - o A GET endpoint to retrieve all service requests
 - o A POST endpoint to add a new request
- Each request should contain title and priority.

Prompt:

Add GET and POST endpoints to manage service requests with title and priority using Flask.

Code:

```
from flask import Flask, jsonify, request

app = Flask(__name__)
# In-memory store for service requests
service_requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({"status": "ok", "message": "API is running!"})

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(service_requests)

@app.route('/requests', methods=['POST'])
def add_request():
    global request_id_counter
    if not request.is_json:
        return jsonify({"error": "Request must be JSON"}), 400

    data = request.get_json()
    title = data.get('title')
    priority = data.get('priority')

    if not title or not priority:
        return jsonify({"error": "Title and priority are required"}), 400

    new_request = {
        'id': request_id_counter,
        'title': title,
        'priority': priority
    }
    service_requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201 # 201 Created

if __name__ == '__main__':
    # Run the app locally. In Colab, you might need to use ngrok for external access.
    # For local testing, you can run this cell and access http://127.0.0.1:5000/status
    app.run(debug=True, port=5000)
```

Output:

```
... * Serving Flask app '__main__'
    * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
    * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Justification:

Implements basic CRUD (Create + Read)

Uses in-memory storage (simple for lab)

Task 3: Modify Existing Resource Using AI Assistance

Scenario

Service requests may change in priority as issues are escalated.

Task Description

- Use AI to generate a PUT endpoint that updates a request based on its index.
- Handle invalid indexes gracefully with proper error responses.
- Ensure the updated request is returned in the response.

Prompt:

Add PUT endpoint to update a request by index with proper error handling in Flask.

Code:

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory store for service requests
service_requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({"status": "ok", "message": "API is running!"})

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(service_requests)

@app.route('/requests', methods=['POST'])
def add_request():
    global request_id_counter
    if not request.is_json:
        return jsonify({"error": "Request must be JSON"}), 400

    data = request.get_json()
    title = data.get('title')
    priority = data.get('priority')

    if not title or not priority:
        return jsonify({"error": "Title and priority are required"}), 400
```

```

new_request = {
    'id': request_id_counter,
    'title': title,
    'priority': priority
}
service_requests.append(new_request)
request_id_counter += 1
return jsonify(new_request), 201 # 201 Created

@app.route('/requests/<int:index>', methods=['PUT'])
def update_request(index):
    if not request.is_json:
        return jsonify({"error": "Request must be JSON"}), 400

    data = request.get_json()
    title = data.get('title')
    priority = data.get('priority')

    if not title and not priority:
        return jsonify({"error": "At least 'title' or 'priority' is required for update"}), 400

    if index < 0 or index >= len(service_requests):
        return jsonify({"error": "Request not found at provided index"}), 404 # Not Found

    req = service_requests[index]
    if title:
        req['title'] = title
    if priority:
        req['priority'] = priority

    return jsonify({"message": "Request updated", "request": req}), 200 # 200 OK

if __name__ == '__main__':
    # Run the app locally. In Colab, you might need to use ngrok for external access.
    # For local testing, you can run this cell and access http://127.0.0.1:5000/status
    app.run(debug=True, port=5000)

```

Output:

```

... To run the app, execute 'python your file name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
* Serving Flask app '__main__'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)

```

Justification:

Enables update operation

Error handling prevents crashes

Task 4: Remove a Resource Safely

Scenario

Resolved service requests must be removed from the active list.

Task Description

- Use AI assistance to implement a DELETE endpoint.
- The endpoint should remove a request by index.

- Return the deleted request details in the response.

Prompt:

Add DELETE endpoint to remove a request by index with safe error handling.

Code:

```
new_request = {
    'id': request_id_counter,
    'title': title,
    'priority': priority
}
service_requests.append(new_request)
request_id_counter += 1
return jsonify(new_request), 201 # 201 Created

@app.route('/requests/<int:index>', methods=['PUT'])
def update_request(index):
    if not request.is_json:
        return jsonify({"error": "Request must be JSON"}), 400

    data = request.get_json()
    title = data.get('title')
    priority = data.get('priority')

    if not title and not priority:
        return jsonify({"error": "At least 'title' or 'priority' is required for update"}), 400

    if index < 0 or index >= len(service_requests):
        return jsonify({"error": "Request not found at provided index"}), 404 # Not Found

    req = service_requests[index]
    if title:
        req['title'] = title
    if priority:
        req['priority'] = priority
```

```

from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory store for service requests
service_requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({"status": "ok", "message": "API is running!"})

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(service_requests)

@app.route('/requests', methods=['POST'])
def add_request():
    global request_id_counter
    if not request.is_json:
        return jsonify({"error": "Request must be JSON"}), 400

    data = request.get_json()
    title = data.get('title')
    priority = data.get('priority')

    if not title or not priority:
        return jsonify({"error": "Title and priority are required"}), 400

    req = {"id": request_id_counter, "title": title, "priority": priority}
    service_requests.append(req)
    request_id_counter += 1

    return jsonify({"message": "Request updated", "request": req}), 200 # 200 OK

@app.route('/requests/<int:index>', methods=['DELETE'])
def delete_request(index):
    if index < 0 or index >= len(service_requests):
        return jsonify({"error": "Request not found at provided index"}), 404 # Not Found

    deleted_request = service_requests.pop(index)
    return jsonify({"message": "Request deleted", "request": deleted_request}), 200 # 200 OK

if __name__ == '__main__':
    # Run the app locally. In Colab, you might need to use ngrok for external access.
    # For local testing, you can run this cell and access http://127.0.0.1:5000/status
    app.run(debug=True, port=5000)

```

Output:

```

... To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
* Serving Flask app '__main__'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)

```

Justification:

- Completes CRUD (Delete)
- Safe removal with validation

Task 5: AI-Assisted API Documentation and Readability

Scenario

Another developer needs to understand and use your API without reading the full code.

Task Description

- Use AI to:
 - o Add meaningful docstrings and inline comments to all endpoints
 - o Clearly describe request methods, inputs, and responses
- Optionally, use AI guidance to integrate automatic API documentation (Swagger or Flask-RESTX).

Prompt:

Add docstrings and comments to Flask API endpoints and optionally integrate Swagger documentation.

Code:

```
from flask import Flask, jsonify, request
from flask_swagger_ui import get_swaggerui_blueprint

app = Flask(__name__)

# In-memory store for service requests
# This list will hold dictionaries, each representing a service request.
service_requests = []
# Counter to assign unique IDs to new requests.
request_id_counter = 1

# --- Swagger UI configuration ---
SWAGGER_URL = '/swagger'
API_URL = '/static/swagger.json' # This would typically be a file or an endpoint serving your OpenAPI spec

# Create a simple OpenAPI spec in memory for demonstration
# In a real application, this would be generated from your code or a separate YAML/JSON file.
swagger_spec = {
    "openapi": "3.0.0",
    "info": {
        "title": "Service Request API",
        "version": "1.0.0",
        "description": "API for managing service requests with title and priority."
    },
    "servers": [
        {
            "url": "http://127.0.0.1:5000",
            "description": "Local Development Server"
        }
    ]
}
```

```

    "paths": {
      "/status": {
        "get": {
          "summary": "Get API status",
          "responses": {
            "200": {
              "description": "API is running",
              "content": {
                "application/json": {
                  "schema": {
                    "type": "object",
                    "properties": {
                      "status": {"type": "string", "example": "ok"},
                      "message": {"type": "string", "example": "API is running!"}
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  },
}

```

```

GET /requests
Retrieves a list of all service requests.
"""

# Returns the entire list of service requests.
return jsonify(service_requests)

@app.route('/requests', methods=['POST'])
def add_request():
    """
    POST /requests
    Adds a new service request to the list.
    Expects a JSON body with 'title' and 'priority'.
    """

    global request_id_counter # Declare global to modify the counter
    # Ensure the request content type is JSON.
    if not request.is_json:
        return jsonify({"error": "Request must be JSON"}), 400

    data = request.get_json()
    title = data.get('title')
    priority = data.get('priority')

    # Validate required fields.
    if not title or not priority:
        return jsonify({"error": "Title and priority are required"}), 400

    # Create a new request dictionary with a unique ID.
    new_request = {
        'id': request_id_counter,
        'title': title.

```



```

# Serve the Swagger JSON directly from an endpoint
@app.route('/static/swagger.json')
def swagger_json():
    return jsonify(swagger_spec)

# Register the Swagger UI blueprint
swaggerui_blueprint = get_swaggerui_blueprint(
    SWAGGER_URL, # Swagger UI endpoint
    API_URL,     # Our API spec
    config={
        'app_name': "Service Request API"
    }
)
app.register_blueprint(swaggerui_blueprint, url_prefix=SWAGGER_URL)
# --- End Swagger UI configuration ---

@app.route('/status')
def status():
    """
    GET /status
    Returns the current status of the API.
    """
    # A simple endpoint to check if the API is running.
    return jsonify({"status": "ok", "message": "API is running!"})

@app.route('/requests', methods=['GET'])
def get_requests():
    """
    service_requests.append(new_request)
    request_id_counter += 1
    # Return the newly created request with a 201 Created status.
    return jsonify(new_request), 201

@app.route('/requests/<int:index>', methods=['PUT'])
def update_request(index):
    """
    PUT /requests/<index>
    Updates an existing service request by its 0-based index.
    Expects a JSON body with 'title' and/or 'priority'.
    """
    # Ensure the request content type is JSON.
    if not request.is_json:
        return jsonify({"error": "Request must be JSON"}), 400

    data = request.get_json()
    title = data.get('title')
    priority = data.get('priority')

    # Require at least one field for update.
    if not title and not priority:
        return jsonify({"error": "At least 'title' or 'priority' is required for update"}), 400

    # Validate the provided index.
    if index < 0 or index >= len(service_requests):
        return jsonify({"error": "Request not found at provided index"}), 404

    # Get the request by index and update its fields.
    req = service_requests[index]

```

```

    if title:
        req['title'] = title
    if priority:
        req['priority'] = priority

    # Return success message and the updated request.
    return jsonify({"message": "Request updated", "request": req}), 200

@app.route('/requests/<int:index>', methods=['DELETE'])
def delete_request(index):
    """
    DELETE /requests/<index>
    Deletes a service request by its 0-based index.
    """
    # Validate the provided index.
    if index < 0 or index >= len(service_requests):
        return jsonify({"error": "Request not found at provided index"}), 404

    # Remove the request from the list.
    deleted_request = service_requests.pop(index)
    # Return success message and the deleted request.
    return jsonify({"message": "Request deleted", "request": deleted_request}), 200

if __name__ == '__main__':
    # When running in Colab, app.run() might not be directly accessible from your browser.
    # For external access, you would typically use a tool like ngrok.
    # For testing within Colab, use the `requests` library to hit the endpoints.
    app.run(debug=True, port=5000)

```

Output:

```

-- To run the app, execute "python your_file_name.py" in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
* Serving Flask app "main"
* Debug mode: on
INFO Werkzeug: WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://0.0.0.0:5000
* Running on http://172.16.0.1:5000
INFO Werkzeug: Press CTRL-C to quit
INFO Werkzeug: * Restarting with watchdog (inotify)

```

Justification:

- Docstrings → improve readability
- Swagger → auto API documentation
- Helps other developers integrate faster